

1. Logika cyfrowa

Układ	Formuła / sygnatura	Opis
Sumatory		
Półsumator	$s = x \oplus y$ $c = x \& y$	Dodaje dwa bity. s to suma, c to bit przeniesienia (carry).
Pełny sumator (FA)	$s = x \oplus y \oplus ci$ $co = x \& y \mid ci \& (x \oplus y)$	Jak półsumator, ale przyjmuje też przeniesienie wejściowe ci .
Sumator n-bitowy	$n \times FA$	Kaskadowo wyjście C_i trafia na wejście kolejnego FA. Ostatnie co to Carry Out całości.

Układy kombinacyjne		
Dekoder	$n \rightarrow 2^n$	Wejście koduje liczbę k . Na wyjściu zapalony jest bit k .
Multiplexer	$2^n + n \rightarrow 1$	2^n bitów danych + n bitów sterujących S . Na wyjściu zapalony S -ty bit danych.
ALU	$A, B, f \rightarrow C$	Dekoder na bitach f wybiera operację np. $A + B$, multipleksowanie wyników bramkami AND/OR.

Układy sekwencyjne (pamięć)		
Przerzutnik S-R	S - set, R - reset	Przechowuje 1 bit (Q). S=1 ustawia $Q = 1$, R=1 zeruje, S=R=0 trzyma stan. Dwa sprzężone NOR -y.
Sterowany poziomem	aktywny gdy CLK = 1	Wejścia S / R zAND-owane z zegarem.
Sterowany zboczem	zbocze 1 $\rightarrow$ 0	Dwa przerzutniki poziomowe w kaskadzie. Stan przepisuje się w momencie zmiany zegara.

2. Typy danych

Typ	char	short	unsigned	int	long	float	double	void*
x86-64	1	2	4	4	8	4	8	8

3. Rozszerzanie, obcinanie i przesunięcia

$x \ll k$	$x \cdot 2^k$ (sgn./uns.)
$u \gg k$	$\lfloor u/2^k \rfloor$ - logiczne (zera)
$x \gg k$	arytmetyczne (kopia MSB), zaokr. do $-\infty$
$x/2^k$ do 0	$(x + (1 \ll (k-1))) \gg k$
Strength reduction	$x * 24 \rightarrow (x \ll 5) - (x \ll 3)$

4. Operacje i endianness

Negacja U2	<code>-x = ~x+1 ; -TMin=TMin , -0=0</code>	
Wykr. nadmiaru (<code>s=x+y</code>)	<code>((s^x)&(s^y))>>(w-1)</code>	
Maska / abs	<code>m=x>>31; abs=(x^m)-m</code>	
<code>c ? a : b</code>	<code>b ^ (-c & (a ^ b))</code>	
promocja w C: <code>unsigned</code> i <code>int</code> w jednym wyrażeniu/porównaniu < > == → <code>int</code> promowany do <code>unsigned</code> (bity bez zmian, -1 → Max).		
Schemat	Najniższy adres	<code>0x0123</code>
Big Endian	MSB	<code>[01][23]</code>
Little Endian	LSB	<code>[23][01]</code>

5. Zmiennoprzecinkowe (IEEE 754)

$V = (-1)^s \times M \times 2^E$				Bias = $2^{k-1} - 1$
Format	bity	s	exp (k)	frac (n)
float	32	1	8	23
double	64	1	11	52
Kategorie wartości				
Kategoria	exp	Własności		
Znormalizowane	!= 0...0 != 1...1	$E = \text{exp} - \text{Bias}$. $M = 1.\text{frac}$. Najgęściej ułożone wokół 0.		
Zdenormaliz.	== 0...0	$E = 1 - \text{Bias}$. $M = 0.\text{frac}$. Reprezentują ± 0.0 (frac = 0) i liczby bardzo bliskie zera.		
Specjalne	== 1...1	frac == 0 $\rightarrow \pm\infty$ (nadmiar, dzielenie przez 0).		
		frac != 0 $\rightarrow$ NaN (np. $\sqrt{-1}$, $\infty -$ ∞).		

Zaokrąglenie GRS i Round-to-Even
Domyślny tryb to Round-to-Even (zapobiega błędowi statycznemu przy sumowaniu).

... x x G ! R S S ... G - ostatni zachowany, R - pierwszy usunięty, S - 0R reszty		
Warunek	Ułamek	Akcja
R=0	< 0.5	W dół (odcięcie)
R=1, S=1	> 0.5	W górę (+1 do G)
R=1, S=0	$= 0.5$	Do parzystej: w górę gdy G=1 , w dół gdy G=0

Własności matematyczne i rzutowanie	
Brak łączności	$(a + b) + c \neq a + (b + c)$ - zaokrąglenia
Brak rozdzielności	$a(b + c) \neq ab + ac$
int $\rightarrow$ float	możliwa utrata precyzji (zaokrągła)
int $\rightarrow$ double	bezstratne ($52 > 32$ bity)
float/double $\rightarrow$ int	obcina do 0; poza zakresem lub NaN $\rightarrow$ TMin (0x80000000)

6. Rejestry x86_64

%rax %eax %ax %ah %al	%rbx %ebx %bx %bh %bl
%rcx %ecx %cx %ch %cl	%rdx %edx %dx %dh %dl
%rsi %esi %si %sil	%rdi %edi %di %dil
%rbp %ebp %bp %bpl	%rsp %esp %sp %spl
%r8 %r8d %r8w %r8b	%r9 %r9d %r9w %r9b

Uwaga: Rejestry od %r10 do %r15 używają schematu:
%r10, %r10d, %r10w, %r10b.

Podział ról rejestrów

%rax	Akumulator. Główny rejestr arytmetyczny. Przechowuje wartość zwracaną.
%rdi, %rsi, %rdx, %rcx, %r8, %r9	Kolejno: od 1. do 6. argumentu funkcji. Caller-saved.
%rsp	Wskaźnik stosu (SP). Wskazuje wierzchołek ramki.
%rbp	Wskaźnik bazy (BP). Początek ramki stosu. Callee-saved.
%rbx, %r12-%r15	Ogólnego przeznaczenia. Wszystkie callee-saved.
%rip	Wskaźnik instrukcji (Program Counter).

Rejestry wektorowe (zmiennoprzecinkowe)

%xmm0- %xmm15	128-bitowe. %xmm0 to wartość zwracana (float / double). Pierwsze 8 to argumenty. Caller-saved.
%ymm0- %ymm15	256-bitowe rozszerzenie XMM. Dzielią z nimi najmłodsze 128 bitów.

7. Tryby adresowania (operandy)

Tryb	Format	Obliczanie adresu
Natychmiastowy (Imm)	\$Imm	Imm
Rejestrowy (Reg)	%Ra	%Ra
Bezpośredni (Mem)	Imm	MCImm
Pośredni (Mem)	C(Rb)	MC(Rb)
Z przesunięciem	D(CRb)	MC(Rb + D)
Skalowany (Ind/scaled)	D(CRb, %Ri, S)	MC(Rb+%Ri*S+D)
Skalowany (baseless)	(, %Ri, S)	MC(Ri * S)

Legenda: Imm / D to stała liczbowa, %Ra / %Rb to rejestr bazowy, %Ri to rejestr indeksowy, a S to skala (tylko: 1, 2, 4 lub 8).

8. Flagi stanu i sterowanie

ZF	Zero. Wynik to 0 (np. argumenty są równe).
SF	Sign. Wynik jest ujemny (MSB = 1).
CF	Carry. Przepełnienie dla liczb bez znaku (unsigned).
OF	Overflow. Przepełnienie dla liczb ze znakiem (signed).

Sufiksy warunkowe (dla jX, setX, cmovX)

Sufiks	Znaczenie
e / z	Equal / Zero (równe / wynik to 0)
ne / nz	Not Equal / Not Zero (nierówne)
s	Sign (wynik ujemny, SF=1)
Liczby ze znakiem (signed)	
g / ge	Greater / Greater or Equal
l / le	Less / Less or Equal
Liczby bez znaku (unsigned)	
a / ae	Above / Above or Equal (No Carry)
b / be	Below / Below or Equal (Carry)

Translacja struktur kontrolnych (C → ASM)

- if-else: jX (skok) lub cmovX (liczy obie ścieżki, bez kary za predykcję). cmovX odpada przy drogich gałęziach, wyłuskaniach (*p) i efektach ubocznych (x++).
- switch: tablica skoków → czas O(1). Skok pośredni: jmp *.L4(%rdi,8). Brak break ⇒ fall-through.
- Pętla for: zawsze redukowana do while: init; while(cond) { body; update; }

Wzorce asemblerowe dla pętli:

do-while	while (Jump-to-mid)	while (Guarded)
loop: Body if(T) goto loop	goto test loop: Body test: if(T) goto loop	if(!T) goto done loop: Body if(T) goto loop done:

9. Procedury i stos

Stos rośnie w dół (niższe adresy); %rsp wskazuje wierzchołek (najniższy zajęty adres). push zmniejsza %rsp o 8, pop zwiększa o 8.

- call dest: odkłada adres powrotu na stos i skacze do dest.
- ret: zdejmuje adres powrotu ze stosu i skacze pod niego.

10. Konwencja Wywoływań (System V ABI)

%rdi → %rsi → %rdx → %rcx → %r8 → %r9

Argumenty 7+: Na stosie od końca. Wynik w %rax.

Rejestry caller-saved vs callee-saved

Caller-saved
(Wołający musi zapisać)
Mogą zostać nadpisane w funkoji.
By je zachować, caller kładzie je na stos przed call.

%rax, %rdi, %rsi, %rdx, %rcx, %r8 - %r11

Callee-saved
(Wołany musi przywrócić)
Muszą zachować stan. Callee musi zapisać je na stos i odtworzyć przed ret.

%rbx, %rbp, %r12 - %r15

Ramka stosu

Każde wywołanie funkcji ma własną ramkę (stąd rekurencja działa naturalnie).

Ramka potrzebna, gdy: brak rejestrów na zmienne (spilling), lokalna tablica/struktura, lub pobrano adres zmiennej (&).

Prolog

```
pushq %rbp ;
zapisz stary base ptr
movq %rsp, %rbp ;
nowy base ptr = rsp
subq $32, %rsp ;
alokacja 32 bajtów
```

Epilog

```
addq $32, %rsp ;
zwolnij alokację
popq %rbp ;
przywróć base ptr
ret ;
skok powrotny
```

`leave` : zastępuje `movq %rbp, %rsp` + `popq %rbp` jedną instrukcją.

11. Architektura CPU

Fazy przetwarzania instrukcji

Fetch → Decode → Execute → Memory → Write

Pipelining i hazardy

Nakładanie faz zwiększa throughput, ale rodzi konflikty (hazardy):

Danych	Odczyt po zapisie - wynik jeszcze nie jest w rejestrze, a kolejna instrukcja już go potrzebuje. Rozwiązanie: forwarding/bypassing (wynik z ALU prosto na wejście) lub stall/bubble.
Sterowania	Skok wymusza odgadnięcie następnego adresu. Rozwiązanie: branch prediction. Pomyłka → czyszczenie potoku (misprediction penalty).

Out-of-Order

- Nowoczesne procesory nie wykonują instrukcji sekwencyjnie:
- Superskalarność: wiele instrukcji na cykl (wiele jednostek wykonawczych).
 - Register renaming: mapowanie rejestrów logicznych (`%rax`) na wiele fizycznych - eliminuje fałszywe zależności.
 - Reorder buffer: instrukcje liczone asynchronicznie, ale zatwierdzone w kolejności programu (spójność).

Miary wydajności

Latency bound	Limit wynikający z łańcucha zależności danych. Zależy od opóźnienia jednostki. Np. wynik z poprzedniej iteracji jest potrzebny w obecnej.
Throughput bound	Limit wynikający z przepustowości sprzętu.

Fazy przetwarzania

Dispatch	Dekodowanie i alokacja w stacji rezerwacyjnej. CPU może zlecić ograniczoną liczbę instrukcji na cykl.
Execute	Wykonywanie właściwe. Zależne instrukcje mogą rozpocząć e w cyklu udostępnienia wyniku (w) przez instrukcję poprzedzającą.
Write-back	Udostępnianie wyniku na magistrali. W tym samym cyklu zależne instrukcje zaczynają e .
Retire	Zatwierdzenie stanu architektury. Musi zachodzić ściśle in-order.

12. Katalog instrukcji (AT&T)

Opcode	Efekt	Opis
<code>mov S, D</code>	$D \leftarrow S$	Kopiuje wartość z S do D.
<code>lea S, D</code>	$D \leftarrow \text{addr}(S)$	Oblicza adres S (bez czytania pamięci).
<code>add S, D</code>	$D \leftarrow D + S$	Dodawanie (ustawia flagi).
<code>sub S, D</code>	$D \leftarrow D - S$	Odejmowanie (ustawia flagi).
<code>imul S, D</code>	$D \leftarrow D * S$	Mnożenie liczb ze znakiem.
<code>sar / shl k, D</code>	$D \ll= k$	Przesunięcie w lewo (mnożenie przez 2^k).
<code>sar k, D</code>	$D \gg= k$	Arytmetyczne w prawo (dzielenie). Kopiuje znak.
<code>shr k, D</code>	$D \gg= k$	Logiczne w prawo. Dopełnia zerami.
<code>and / or S, D</code>	$D \leftarrow D \& / S$	Operacje bitowe (ustawiają flagi).
<code>xor S, D</code>	$D \leftarrow D \wedge S$	Często jako <code>xor %rax, %rax</code> do zerowania.

Porównania i sterowanie

<code>cmp S1, S2</code>	$S2 - S1$	Ustawia flagi jak odejmowanie, nie zapisuje wyniku.
<code>test S1, S2</code>	$S2 \& S1$	Ustawia flagi jak AND.
<code>jX dest</code>	$\text{if}(X) \%rip \leftarrow \text{dest}$	Skok warunkowy (X = warunek).
<code>setX D</code>	$\text{if}(X) D \leftarrow 1$	Ustawia najmłodszy bajt na 0 lub 1 wg flag.
<code>cmovX S, D</code>	$\text{if}(X) D \leftarrow S$	Warunkowe kopiowanie (zamiast skoku).

Stos i zarządzanie procedurami

<code>push S</code>	$\%rsp - = 8$ $M[\%rsp] \leftarrow S$	Odkłada na stos (zmniejsza <code>%rsp</code>).
<code>pop D</code>	$D \leftarrow M[\%rsp]$ $\%rsp + = 8$	Zdejmuje ze stosu do D (zwiększa <code>%rsp</code>).
<code>call dest</code>	$\text{push } \%rip$ $\%rip \leftarrow \text{dest}$	Skok do funkcji (zapisuje adres powrotu na stosie).
<code>ret</code>	$\text{pop } \%rip$	Zdejmuje adres powrotu ze stosu i skacze pod niego.
<code>leave</code>	$\%rsp \leftarrow \%rbp$ $\text{pop } \%rbp$	Sprząta ramkę stosu (odwrotność prologu).

<code>vmovaps / ups S, D</code>	$D \leftarrow S$	Kopiuje wektor. a (aligned - szybkie, adres podzielny przez wyrównanie), u (unaligned).
<code>vaddps / pd S1, S2, D</code>	$D \leftarrow S2 + S1$	Dodawanie wielokrotne (packed). ps (single-float), pd (double).
<code>vmulps / pd S1, S2, D</code>	$D \leftarrow S2 * S1$	Mnożenie wektorowe.
<code>vfmadd231ps S1, S2, D</code>	$D \leftarrow S2 * S1 + D$	Fused Multiply-Add. Mnoży i dodaje do D w jednym kroku.

Uwaga o sufiksach: Instrukcje przyjmują sufiks określający rozmiar danych: **b** (8-bit), **w** (16-bit), **l** (32-bit), **q** (64-bit).
Np. `movl` kopiuje 32 bity (i automatycznie zeruje górną połowę 64-bitowego rejestru!).